

Scripst 的使用方法

article 样式

AnZrew

AnZreww

AnZrewww

2026-08-18

摘要: Scripst 是一款简约易用的 Typst 语言模板，适用于日常文档、作业、笔记、论文等多种场景

关键词: Scripst; Typst; 模板

目录

I	使用 Scripst 排版 Typst 文档	3
I.1	使用 Typst	3
I.2	使用 Scripst	3
I.2.1	在线使用	4
I.2.2	离线使用	4
II	模板参数说明	5
II.1	template	6
II.2	title	6
II.3	info	7
II.4	author	7
II.5	time	7
II.6	abstract	7
II.7	keywords	8
II.8	font-size	8
II.9	contents	8
II.10	content-depth	9
II.11	matheq-depth	9
II.12	counter-depth	9
II.13	cb-counter-depth	10
II.14	countblocks	10
II.15	matheq-outline	10

II.16 counter-outline	10
II.17 matheq-color	10
II.18 counter-color	10
II.19 header	11
II.20 lang	11
II.21 par-indent	11
II.22 par-leading	12
II.23 par-spacing	12
II.24 numbering-format	12
II.25 chapter-numbering-format	13
II.26 offset	13
II.27 body	14
III 模板效果展示	14
III.1 扉页	14
III.2 目录	14
III.3 文字样式与环境	14
III.3.1 字体	14
III.3.2 环境	14
III.3.3 列举	16
III.3.4 引用	16
III.3.5 链接	16
III.3.6 超链接与文献引用	17
III.4 #newpara()函数	17
III.5 labelset	17
III.6 countblock	18
III.6.1 默认提供的 countblock	18
III.6.2 调整所有 countblock 的深度	22
III.6.3 调整个别 countblock 的深度	22
III.6.4 增添新的 countblock	23
III.6.5 cb 字典结构	24
III.6.6 countblock 的使用	24
III.6.7 总结	26
III.7 一些其他的块	26
III.7.1 空白块	26
III.7.2 证明与■（证明结束）	27

III.7.3 解答	27
III.7.4 分隔符	27
III.8 其他格式调整	27
III.8.1 数学公式相关	27
IV 结语	28

Typst 是一种简单的文档生成语言，它的语法类似于 Markdown 的轻量级标记，利用合适的 `set` 和 `show` 指令，可以高自由度地定制文档的样式。

Scripst 是一款简约易用的 Typst 语言模板，适用于日常文档、作业、笔记、论文等多种场景。

I 使用 Scripst 排版 Typst 文档

I.1 使用 Typst

Typst 是使用起来比 LaTeX 更轻量的语言，一旦模板编写完成，就可以用类似 Markdown 的轻量标记完成文档的编写。

相比 LaTeX，Typst 的优势在于：

- 极快的编译速度
- 语法简单、轻量
- 代码可扩展性强
- 更简单的数学公式输入
- ...

所以，Typst 非常适合轻量级日常文档的编写。只需要花费撰写 Markdown 的时间成本，就能获得甚至比 LaTeX 还要好的排版效果。

可以通过下面的方式安装 Typst：

```
sudo apt install typst # Debian/Ubuntu
sudo pacman -S typst # Arch Linux
winget install --id Typst.Typst # Windows
brew install typst # macOS
```

你也可以在 [Typst 的 GitHub 仓库](#) 中找到更多的信息。

I.2 使用 Scripst

在 Typst 的基础上，Scripst 提供了一些简约的，可便利日常文档生成的模板样式。

I.2.1 在线使用

[Scripst 包](#)已经提交至社区，在联网的状态下，可直接使用

```
#import "@preview/scripst:1.1.2": *
```

来引入 Scripst 的模板。你也可以通过 `typst init` 来一键使用模板创建新的项目：

```
typst init @preview/scripst:1.1.2 project_name
```

这种方法无需手动下载模板文件，只需要在文档中引入即可。

I.2.2 离线使用

解压使用

可以在 [Scripst 的 GitHub 仓库](#)找到并下载 Scripst 的模板。

可以选择 `<> code` → `Download ZIP` 来下载 Scripst 的模板。在使用时，只需要将模板文件放在你的文档目录下，然后在文档的开头引入模板文件即可。



要考虑清楚项目的目录结构，以便正确引入模板文件。

```
project/
├── src/
│   ├── main.typ
│   ├── ...
│   └── components.typ
├── pic/
│   └── ...
├── main.typ
├── chap1.typ
├── chap2.typ
└── ...
```

如果项目的目录结构如上所示，那么在 `main.typ` 中引入模板文件的方式应该是：

```
#import "src/main.typ": *
```

这种方法的好处是，你可以随时调整模板中的部分参数。Script 模板采用模块化设计，你可以轻松找到并修改模板中你需要修改的部分。

本地包管理

一个更好的方法是，参考官方给出的[本地的包管理文档](#)，将模板文件放在本地包管理的目录`{data-dir}/typst/packages/{namespace}/{name}/{version}`下，这样就可以在任何地方使用 Scripst 的模板了。

当然，无需担心模板文件难以修改，你可以直接在文档中使用 `#set`，`#show` 等指令来覆盖模板中的部分参数。

例如该模板的应该放在

```
~/ .local/share/typst/packages/preview/scripst/1.1.2      # in Linux
%APPDATA%\typst\packages\preview\scripst\1.1.2          # in Windows
~/Library/Application Support/typst/packages/local/scripst/1.1.2 # macOS
```

你可以执行指令

```
cd ~/.local/share/typst/packages/preview/scripst
git clone https://github.com/An-314/scripst.git 1.1.2
```

如果是这样的目录结构，那么在文档中引入模板文件的方式应该是：

```
#import "@preview/scripst:1.1.2": *
```

这样的好处是你可以直接通过 `typst init` 来一键使用模板创建新的项目：

```
typst init @preview/scripst:1.1.2 project_name
```

在引入模板后通过这样的方式创建一个 `article` 文件：

```
#show: scripst.with(
  title: [Scripst 的使用方法],
  info: [这是文章的模板],
  author: ("作者1", "作者2", "作者3"),
  time: datetime.today().display(),
  abstract: [摘要],
  keywords: ("关键词1", "关键词2", "关键词3"),
  contents: true,
  content-depth: 2,
  matheq-depth: 2,
  lang: "zh",
)
```

这些参数以及其含义见 [小节 II](#)。

这样你就可以开始撰写你的文档了。

II 模板参数说明

Scriptst 的模板提供了一些参数，用来定制文档的样式。

```
#let scriptst(
  template: "article",           // str: ("article", "book", "report")
  title: "",                     // str, content, none
  info: "",                      // str, content, none
  author: (),                   // str, content, array, none
  time: "",                     // str, content, none
  abstract: none,               // str, content, none
  keywords: (),                 // array
  font-size: 11pt,              // length
  contents: false,              // bool
  content-depth: 2,             // int
  matheq-depth: 2,              // int: (1, 2, 3)
  counter-depth: 2,             // int: (1, 2, 3)
  cb-counter-depth: 2,          // int: (1, 2, 3)
  countblocks: cb,              // dict
  matheq-outline: "(1.1)",       // str, function
  counter-outline: "1.1",        // str, function
  header: true,                 // bool
  lang: "zh",                   // str: ("zh", "en", "fr", ...)
  par-indent: 2em,              // length
  par-leading: none,            // length
  par-spacing: none,            // length
  numbering-format: "I.1",       // str, function
  chapter-numbering-format: "I", // str, function
  offset: 0,                    // int
  body,
) = {
  ...
}
```

II.1 template

参数	类型	可选值	默认值	说明
template	str	("article", "book", "report")	"article"	模板类型

目前 Scriptst 提供了三种模板，分别是 article、book 和 report。

本模板采用 article 模板。

- article: 适用于日常文档、作业、小型笔记、小型论文等
- book: 适用于书籍、课程笔记等
- report: 适用于实验报告、论文等

此外的字符串传入会导致 panic: "Unknown template!"。

II.2 title

参数	类型	默认值	说明
title	content, str, none	""	文档标题

文档的标题。（不为空时）会出现在文档的开头和页眉中。

II.3 info

参数	类型	默认值	说明
info	content, str, none	""	文档信息

文档的信息。（不为空时）会出现在文档的开头和页眉中。可以作为文章的副标题或者补充信息。

II.4 author

参数	类型	默认值	说明
author	str, content, array, none	()	文档作者

文档的作者。要传入 `str` 或者 `content` 的列表，或者直接的 `str` 或者 `content` 对象。

Note

注意，如果是一个作者的情况，可以只传入 `str` 或者 `content`，在多个作者的时候传入一个 `str` 或者 `content` 的列表，例如：`author: ("作者 1", "作者 2")`

会在文章的开头以 `min(#authors, 3)` 个作为一行显示。

II.5 time

参数	类型	默认值	说明
time	content, str, none	""	文档时间

文档的时间。会出现在文档的开头和页眉中。

你可以选择用 `typst` 提供的 `datetime` 来获取或者格式化时间，例如今天的时间：

```
datetime.today().display()
```

II.6 abstract

参数	类型	默认值	说明
abstract	content, str, none	none	文档摘要

文档的摘要。（不为空时）会出现在文档的开头。

建议在使用摘要前，首先定义一个 `content`，例如：

```
#let abstract = [
    这是一个简单的文档模板，用来生成简约的日常使用的文档，以满足文档、作业、笔记、论文等
    需求。
]

#show: scripst.with(
    ...
    abstract: abstract,
    ...
)
```

然后将其传入 `abstract` 参数。

II.7 keywords

参数	类型	默认值	说明
keywords	array	()	文档关键词

文档的关键词。要传入 `str` 或者 `content` 的列表。

和 `author` 一样，参数是一个列表，而不能是一个字符串。

只有在 `abstract` 不为空时，关键词才会出现在文档的开头。

II.8 font-size

参数	类型	默认值	说明
font-size	length	11pt	文档字体大小

文档的字体大小。默认为 `11pt`。

参考 `length` 类型的值，可以传入 `pt`、`mm`、`cm`、`in`、`em` 等单位。

II.9 contents

参数	类型	默认值	说明
contents	bool	false	是否生成目录

是否生成目录。默认为 `false`。

II.10 content-depth

参数	类型	默认值	说明
content-depth	int	2	目录的深度

目录的深度。默认为 2。

II.11 matheq-depth

参数	类型	可选值	默认值	说明
matheq-depth	int	1, 2, 3	2	数学公式的深度

数学公式编号的深度。默认为 2。

Note

计数器的详细表现见 [小节 II.12](#)。

II.12 counter-depth

参数	类型	可选值	默认值	说明
counter-depth	int	1, 2, 3	2	计数器的深度

文中 `figure` 环境中的图片 `image`，表格 `table`，以及代码 `raw` 的计数器深度。默认为 2。

Note 计数器的详细表现

一个计数器的深度为 1 时，计数器的编号会是全局的，不会受到章节的影响，即 1, 2, 3, ...。

一个计数器的深度为 2 时，计数器的编号会受到一级标题的影响，即 1.1, 1.2, 2.1, 2.2, ...。但如果此时整个文档没有一级标题，Scripst 会自动将其转化为深度为 1 的情况。

一个计数器的深度为 3 时，计数器的编号会受到一级标题和二级标题的影响，即 1.1.1, 1.1.2, 1.2.1, 1.2.2, 2.1.1, ...。但如果此时整个文档没有二级标题但有一级标题，Scripst 会自动将其转化为深度为 2 的情况；如果没有一级标题，Scripst 会自动将其转化为深度为 1 的情况。

II.13 cb-counter-depth

参数	类型	可选值	默认值	说明
cb-counter-depth	int	1, 2, 3	2	countblock 的计数器深度

通过 countblocks 传入的 countblock 的默认编号深度，默认为 2。使用 set-countblock-depth 或 add-countblock(depth: ...) 指定的个别深度优先于该默认值。详情见 [小节 III.6.2](#)。

II.14 countblocks

模板使用的 countblock 字典，默认为 cb。创建自定义 countblock 后，应当在 #show: scripst.with(...) 中通过该参数传入更新后的字典，以便 Ratchet 配置其编号和引用。

II.15 matheq-outline

参数	类型	可选值	默认值	说明
matheq-outline	str, func	(1.1), (A.1), ...	(1.1)	数学公式的编号格式

公式编号的编号格式。默认为 "(1.1)"，即 (1.1)、(1.2)、(2.1)、(2.2) 等。

II.16 counter-outline

参数	类型	可选值	默认值	说明
counter-outline	str, func	1.1, A.1, ...	1.1	计数器的编号格式

文中 figure 环境中的图片 image，表格 table，以及代码 raw 的编号格式。默认为 "1.1"，即 1.1、1.2、2.1、2.2 等。

II.17 matheq-color

参数	类型	可选值	默认值	说明
matheq-color	color	black, blue, red, ...	red	数学公式引用的颜色

数学公式引用的颜色。默认为 "red"。

II.18 counter-color

参数	类型	可选值	默认值	说明
counter-color	color	black, blue, red, ...	blue	计数器引用的颜色

计数器引用的颜色。默认为"blue"。

II.19 header

参数	类型	默认值	说明
header	bool	true	页眉

是否生成页眉。默认为 true。

Note

页眉包括文档的题目、信息和当前所在的章节标题。

- 如果三者都存在，则将会三等分地显示在页眉中。
- 如果文档没有信息，页眉仅会在最左最右显示文档的题目和当前所在的章节标题。
 - 进而如果文档没有题目，页眉仅会在最右侧显示当前所在的章节标题。
- 如果文档没有任何一级标题，页眉将仅在最左最右显示文档的题目和信息。
 - 进而如果文档没有信息，页眉仅会在最左侧显示文档的题目。
- 如果什么都没有，页眉将不会显示。

II.20 lang

参数	类型	默认值	说明
lang	str	"zh"	文档语言

文档的语言，默认为"zh"。

接受 [ISO_639-1](#) 编码格式传入，如"zh"、"en"、"fr"等。

II.21 par-indent

参数	类型	默认值	说明
par-indent	length	2em	段落首行缩进

段落首行缩进。默认为 2em。如果调节成 0em，则为不缩进。

II.22 par-leading

参数	类型	默认值	说明
par-leading	length	跟随 lang 设置	段落内的行间距

段落内间距。在中文文档中默认是 **1em**。

Note

默认值会随着语言的选择而变化，具体情况见下表

语言类型	默认值
东亚文字（汉语、韩语、日语等）	1em
南亚、东南亚、阿姆哈拉文字（泰语、越南语、缅甸语、印地语、阿姆哈拉语等）	0.85em
阿拉伯文字（阿拉伯语、波斯语等）	0.75em
斯拉夫文字（俄语、保加利亚语等）	0.7em
其他文字	0.6em

II.23 par-spacing

参数	类型	默认值	说明
par-spacing	length	跟随 lang 设置	段落间距

段落间距。在中文文档中默认是 **1.2em**。

Note

默认值会随着语言的选择而变化，具体情况见下表

语言类型	默认值
东亚文字（汉语、韩语、日语等）	1.2em
南亚、东南亚、阿姆哈拉文字（泰语、越南语、缅甸语、印地语、阿姆哈拉语等）	1.3em
阿拉伯文字（阿拉伯语、波斯语等）	1.25em
斯拉夫文字（俄语、保加利亚语等）	1.2em
其他文字	1em

II.24 numbering-format

参数	类型	默认值	说明
numbering-format	str, function	"1.1"	章节标题的编号格式

章节标题的编号格式。默认为"1.1"，即 1.1、2.1、3.1 等。

你可以传入一个函数来实现自定义的编号格式，例如：

```
#let numbering = (n, ..it) => "Chapter" + str(n) + " " + numbering("1.1", ..it)
```

Note

- 如果传入字符串，则会以其为 pattern 来格式化章节标题的编号，并且根据 offset 设置标题编号的偏移。
- 如果传入函数，则会忽略掉 offset 的设置，一切按照函数的逻辑来进行编号。

II.25 chapter-numbering-format

参数	类型	默认值	说明
chapter-numbering-format	str, function	根据语言而定	章节标题的编号格式

Note

目前 typst 如下的 bug，当你以字符串的形式传入 chapter-numbering-format 时，如果字符串内含"i"，typst 会自动匹配当成罗马数字的格式来处理，例如"i"、"ii"、"iii"等。这个问题无法通过转义来解决。

所以当你需要传入含有 i 的 pattern 时，建议使用函数的方式来传入，例如：

```
#let numbering = (n, ..it) => "Unit" + str(n)
```

这样就可以避免 typst 的 bug 了。

II.26 offset

参数	类型	默认值	说明
offset	int	0	章节标题的偏移

章节标题的偏移。默认为 0。文档内的第一节会以 1 + offset 开始编号。所以 offset 的最小值为-1，此时文档的第一节的编号是 0。

II.27 body

在使用 `#show: scripst.with(...)` 时, `body` 参数是不用手动传入的, `typst` 会自动将剩余的文档内容传入 `body` 参数。

III 模板效果展示

III.1 扉页

文档的开头会显示标题、信息、作者、时间、摘要、关键词等信息, 如该文档的扉页所示。

III.2 目录

如果 `contents` 参数为 `true`, 则会生成目录, 效果见本文档目录。

III.3 文字样式与环境

Scripst 提供了一些常用的文字样式和环境, 如粗体、斜体、标题、图片、表格、列表、引用、链接、数学公式等。

III.3.1 字体

这是正常的文本。 This is a normal text.

这是粗体的文本。 **This is a bold text.**

这是斜体的文本。 *This is an italic text.*

安装 CMU Serif 字体以获得更好（类似 LaTeX）的显示效果。

III.3.2 环境

III.3.2.1 标题

一级标题编号随文档语言而异, 包括中文/罗马数字/希腊字母/假名/阿拉伯文数字/印地文数字等, 其余级别标题采用阿拉伯数字编号。

III.3.2.2 图片

图片环境会自动编号, 如下所示:

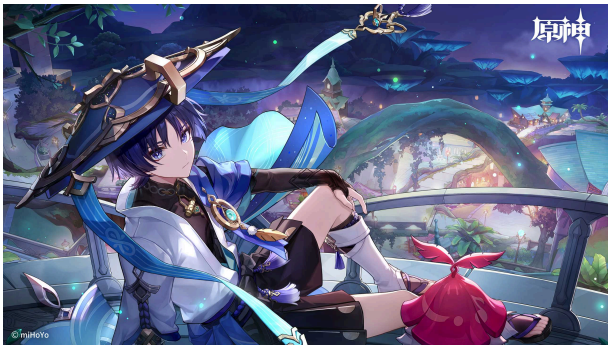


图 3.3.1 散宝

III.3.2.3 表格

得益于 `tablem` 包，使用本模板时可以用 Markdown 的方式编写表格，如下所示：

```
#figure(  
  three-line-table[  
    | 姓名 | 年龄 | 性别 |  
    | --- | --- | --- |  
    | 张三 | 18 | 男 |  
    | 李四 | 19 | 女 |  
  ],  
  caption: [ `three-line-table` 表格示例 ],  
)
```

姓名	年龄	性别
张三	18	男
李四	19	女

表 3.3.1 `three-line-table` 表格示例

```
#figure(  
  tablem[  
    | 姓名 | 年龄 | 性别 |  
    | --- | --- | --- |  
    | 张三 | 18 | 男 |  
    | 李四 | 19 | 女 |  
  ],  
  caption: [ `tablem` 表格示例 ],  
)
```

姓名	年龄	性别
张三	18	男
李四	19	女

表 3.3.2 `tablem` 表格示例

可以选择 `numbering: none`,使得表格不编号，如上所示，前面章节的表格并没有进入全文的表格计数器。

III.3.2.4 数学公式

数学公式有行内和行间两种模式。

行内公式： $a^2 + b^2 = c^2$ 。

行间公式：

$$a^2 + b^2 = c^2$$

$$\frac{1}{2} + \frac{1}{3} = \frac{5}{6} \quad (3.3.1)$$

是拥有编号的。

得益于 `physica` 包，`typst` 本身简单的数学输入方式得到了极大的扩展，并且仍然保留简洁的特性：

$$\begin{aligned} \nabla \cdot \boldsymbol{E} &= \frac{\rho}{\varepsilon_0} \\ \nabla \cdot \boldsymbol{B} &= 0 \\ \nabla \times \boldsymbol{E} &= -\frac{\partial \boldsymbol{B}}{\partial t} \\ \nabla \times \boldsymbol{B} &= \mu_0 \left(\boldsymbol{J} + \varepsilon_0 \frac{\partial \boldsymbol{E}}{\partial t} \right) \end{aligned} \quad (3.3.2)$$

III.3.3 列举

`typst` 为列举提供了简单的环境，如所示：

- 第一项
- 第二项
- 第三项

- 第一项
- 第二项
- 第三项

- + 第一项
- 3. 第二项
- + 第三项

1. 第一项
3. 第二项
4. 第三项

- / 第一项：1
- / 第二项：2
- / 第三项：3

- 第一项 1
- 第二项 2
- 第三项 3

III.3.4 引用

```
#quote(attribution: "爱因斯坦", block:
true)[
上帝不会掷骰子。
]
```

上帝不会掷骰子。

— 爱因斯坦

III.3.5 链接

```
#link("https://www.google.com/")
[Google]
```

Google

III.3.6 超链接与文献引用

利用`<label>`和`@label` 可以实现超链接和文献引用。

III.4 #newpara()函数

默认某些模块不自动换行。这是有必要的，例如，数学公式后面如果不换行就表示对上面的数学公式的解释。

但有时候我们需要换行，这时候就可以使用 `#newpara()` 函数。

区别于官方提供的 `#parbreak()` 函数，`#newpara()` 函数会在段落之间插入一个空行，这样无论在什么场景下，都会开启新的自然段。

只要你觉得需要换行，就可以使用 `#newpara()` 函数。

III.5 labelset

得益于 `typst` 中的 `label` 函数，除了给这种类型添加标签外，还可以通过 `label` 方便地为所引用的对象设置样式。

因此，`Scripst` 内置了一些常用的设置，你可以通过直接添加 `label` 来设置样式。

```
== Schrödinger equation <hd.x>
```

下面是 Schrödinger 方程：

```
$
i \hbar \frac{d}{dt} \ket{\Psi(t)} = \hat{H} \ket{\Psi(t)}
```

```
<text.blue>
```

其中

```
$
\ket{\Psi(t)} = \sum_n c_n \ket{\phi_n}
```

```
<eq.c>
```

是波函数。由此可以得到定态的 Schrödinger 方程：

```
$
\hat{H} \ket{\Psi(t)} = E \ket{\Psi(t)}
```

```
$
```

```
<text.teal>
```

其中 E 是#[能量]。

Schrödinger equation

下面是 Schrödinger 方程：

$$i\hbar \frac{d}{dt}|\Psi(t)\rangle = \hat{H}|\Psi(t)\rangle$$

(3.6.1)

其中

$$|\Psi(t)\rangle = \sum_n c_n |\varphi_n\rangle$$

是波函数。由此可以得到定态的 Schrödinger 方程：

$$\hat{H}|\Psi(t)\rangle = E|\Psi(t)\rangle$$

(3.6.2)

其中 E 是能量。

目前 Scripst 提供了以下的设置：

标签	功能
eq.c	给数学环境的公式取消编号
hd.c	给标题取消编号，但还在目录中显示
hd.x	给标题取消编号，且不在目录中显示
	给文本设置颜色
text.{color}	color in (black, gray, silver, white, navy, blue, aqua, teal, eastern, purple, fuchsia, maroon, red, orange, yellow, olive, green, lime,)

表 3.6.1 Label Set



上述字符串已关联特定样式，允许进行样式覆盖，但在调用 label 和 reference 方法时，请保留这些字符串的原始定义。

III.6 countblock

Definition 3.1 countblock

Countblock 是 Scripst 提供的一个计数器模块，用来对文档中的某些可以计数的内容进行计数。

现在你看到的就是一个 definition 块，它是一个计数器模块的例子。

III.6.1 默认提供的 countblock

Scripst 默认提供如下 countblock。表中的深度 2 表示跟随一级标题编号；它们都继承全局参数 cb-counter-depth: 2。

块名称	cb 名称	counter-name	默认深度	颜色	调用函数
Definition	def	def	2	mycolor.green	#definition
Theorem	thm	thm	2	mycolor.blue	#theorem
Proposition	prop	prop	2	mycolor.violet	#proposition
Lemma	lem	prop	2	mycolor.violet-light	#lemma
Corollary	cor	prop	2	mycolor.violet-dark	#corollary
Remark	rmk	prop	2	mycolor.violet-darker	#remark
Claim	clm	prop	2	mycolor.violet-deep	#claim
Exercise	ex	ex	2	mycolor.purple	#exercise
Problem	prob	prob	2	mycolor.orange	#problem
Example	eg	eg	2	mycolor.cyan	#example
Note	note	note	2 (默认不计数)	mycolor.grey	#note
Caution	cau	cau	2 (默认不计数)	mycolor.red	#caution

Scriptst 默认 countblock 配置

proposition、lemma、corollary、remark 和 claim 的 counter-name 都是 "prop"，因此默认共享同一列编号和同一种重置深度；它们的标题和颜色仍然各自独立。

这些函数的参数和效果是一样的，只是计数器的名称不同。

```
#definition(
  subname: [],
  count: true,
  lab: none,
)[
  ...
]
```

参数说明如下

参数	类型	默认值	说明
subname	array	[]	该条目的名称
count	bool	true	是否计数
lab	str	none	该条目的标签

下面是一个示例：

```
#theorem(subname: [_Fermat's Last Theorem_], lab: "fermat")
```

```
No three $a, b, c$ in  $\mathbb{N}^+$  can satisfy the equation

$$a^n + b^n = c^n$$

for any integer value of  $n$  greater than 2.
]
#proof[Cuius rei demonstrationem mirabilem sane detexi. Hanc marginis
exiguitas non caperet.]
```

就会创建一个定理块，并且计数：

Theorem 3.1 *Fermat's Last Theorem*

No three $a, b, c \in \mathbb{N}^+$ can satisfy the equation

$$a^n + b^n = c^n \tag{3.7.1}$$

for any integer value of n greater than 2.

Proof. Cuius rei demonstrationem mirabilem sane detexi. Hanc marginis exiguitas non caperet.

■

III.6.1.1 subname 参数

`subname` 是会显示在计数器后的信息，例如定理名称等。在上述例子中是“Fermat's Last Theorem”。

III.6.1.2 lab 参数

此外，你可以使用 `lab` 参数来为这个块添加一个标签，以便在文中引用。例如刚才的 `fermat` 定理块，你可以使用 `@fermat` 来引用它。

Fermat 并没有对 `@fermat` 给出公开的证明。

Fermat 并没有对 [Theorem 3.1](#) 给出公开的证明。

在默认提供的这些块中，`proposition`, `lemma`, `corollary`, `remark`, `claim`, 是共用同一个计数器的，效果如下：

Lemma 3.1

这是一个引理，请你证明它。

Proposition 3.2

这是一个命题，请你证明它。

Corollary 3.3

这是一个推论，请你证明它。

Remark 3.4

这是一个评论，请你注意它。

Claim 3.5

这是一个断言，请你证明它。

而其余的计数器是互相独立的。

III.6.1.3 count 参数

此外，对于 `count` 参数，如果你不想计数，可以将其设置为 `false`。`note` 和 `caution` 默认不计数。如果你想要计数，可以将其设置为 `true`。

```
#note(count: true)[
```

这是一个注记，请你注意它。

```
]
```

```
#note[
```

这是一个注记，请你注意它。

```
]
```

Note 3.1

这是一个注记，请你注意它。

Note

这是一个注记，请你注意它。

III.6.2 调整所有 countblock 的深度

`cb-counter-depth` 是所有 `countblock` 的全局默认深度，可取 1、2 或 3：

- 1: 全文连续编号，如 1, 2, 3;
- 2: 随一级标题重置，如 1.1, 1.2, 2.1;
- 3: 随一级、二级标题重置，如 1.1.1, 1.1.2, 1.2.1。

例如，把所有没有单独指定深度的 `countblock` 调整为深度 3：

```
#show: scripst.with(
  countblocks: cb,
  cb-counter-depth: 3,
)
```

`cb-counter-depth` 只改变默认值。已经在 `countblocks` 中单独指定深度的计数器族不会受它影响。

III.6.3 调整个别 countblock 的深度

使用 `set-countblock-depth` 可以覆盖某个计数器族的深度：

```
#let blocks = set-countblock-depth(cb, "thm", 3)

#show: scripst.with(
  countblocks: blocks,
  cb-counter-depth: 2,
)
```

此时只有 `theorem` 使用深度 3，其余独立计数器仍继承全局深度 2。

函数签名如下：

```
#set-countblock-depth(cb, name, depth, detach: false)
```

参数	类型	默认值	说明
<code>cb</code>	<code>dict</code>		原 <code>countblock</code> 字典
<code>name</code>	<code>str</code>		要调整的 <code>cb</code> 名称
<code>depth</code>	<code>int</code>		新深度，只能是 1、2 或 3
<code>detach</code>	<code>bool</code>	<code>false</code>	是否从原共享计数器中拆出该块

III.6.3.1 调整共享计数器族

`proposition`、`lemma`、`corollary`、`remark` 和 `claim` 共用 `counter-name: "prop"`。因此下面的写法会把整个共享族一起改为深度 3：

```
#let blocks = set-countblock-depth(cb, "lem", 3)
```

这是必要的：共享同一个计数器的块必须在相同标题位置一起重置，不能同时拥有不同深度。

III.6.3.2 将一个块拆成独立深度

如果只希望 lemma 使用深度 1，同时让其他 prop 族成员继续使用深度 2，设置 detach: true:

```
#let blocks = set-countblock-depth(cb, "lem", 1, detach: true)
#let lemma = countblock.with("lem", blocks)

#show: scripst.with(
  countblocks: blocks,
  cb-counter-depth: 2,
)
```

detach: true 会把 lemma 的 counter-name 从 "prop" 改成 "lem"。因为计数器身份发生了变化，所以需要更新后的 blocks 重新封装 lemma。只调整本来就独立的 theorem、definition 等块的深度时，不需要重新封装默认函数。

III.6.4 增添新的 countblock

使用 add-countblock 添加新块，并把更新后的字典传给 scripst(countblocks: ...):

```
#let blocks = add-countblock(
  cb,
  "alg",
  "Algorithm",
  yellow,
  depth: 3,
)
#let algorithm = countblock.with("alg", blocks)

#show: scripst.with(
  countblocks: blocks,
  cb-counter-depth: 2,
)
```

此时 algorithm 使用独立计数器和深度 3；其他默认块仍使用全局深度 2。

add-countblock 的完整签名如下：

```
#add-countblock(cb, name, info, color, counter-name: none, depth: none)
```

参数	类型	默认值	说明
cb	dict		原 countblock 字典
name	str		新块在字典中的名称
info	str 或 content		显示在块标题中的名称
color	color		背景和左边框颜色
counter-name	str	none	实际计数器名；默认与 name 相同
depth	int 或 none	none	独立深度；none 表示继承 cb-counter-depth

若要让新块共享已有编号，可以指定已有的 counter-name。例如让 Assumption 加入 theorem 的编号序列：

```
#let blocks = add-countblock(
  cb,
  "asm",
  "Assumption",
  aqua,
  counter-name: "thm",
)
#let assumption = countblock.with("asm", blocks)
```

共享 counter-name 的所有条目必须使用相同深度；否则 Scripst 会直接报错，避免出现显示编号和重置规则不一致。

III.6.5 cb 字典结构

每一项的结构是 (info, color, counter-name, depth)。第四项 depth 可以省略或设为 none，表示继承全局 cb-counter-depth：

```
#let cb = (
  "def": ("Definition", mycolor.green, "def", none),
  "thm": ("Theorem", mycolor.blue, "thm", none),
  "prop": ("Proposition", mycolor.violet, "prop", none),
  "lem": ("Lemma", mycolor.violet-light, "prop", none),
  // ...
  "cb-counter-depth": 2,
)
```

III.6.6 countblock 的使用

定义好一个块之后，就可以使用 countblock 函数来创建它：

```
#countblock(
  name,
  cb,
  subname: "",
```

```
count: true,
lab: none
)[
...
]
```

参数说明如下

参数	类型	默认值	说明
name	str		计数器的名称
cb	dict		计数器字典
subname	str		该条目的名称
count	bool	true	是否计数
lab	str	none	该条目的标签

- name 是计数器的名称，也就是在 `add-countblock` 中显示指定的参数。
- cb 是一个字典，其格式如[小节 III.6.5](#) 所示。注意，你需要传含有该计数器的（最新的）cb，所以一定需要先更新 cb，再传入。
- subname 是会显示在计数器后的信息，例如定理名称等。
- count 是一个布尔值，如果你不想计数，可以将其设置为 `false`。
- lab 是一个字符串，如果你想要为这个块添加一个标签，以便在文中引用，可以使用这个参数。

例如，使用在 [小节 III.6.4](#) 中创建的 test:

```
#countblock("test", blocks)[
  1 + 1 = 2
]
```

This is a test 3.1

1 + 1 = 2

当然也可以将其封装成另一个函数：

```
#let test = countblock.with("test", blocks)
```

然后使用 test 函数：

```
#test[
  1 + 1 = 2
]
```

This is a test 3.2

$1 + 1 = 2$

III.6.7 总结

Script 通过 `add-countblock` 扩展字典，通过 `countblocks` 将整个字典一次性交给模板，再通过 `countblock` 创建具体的块。编号、重置和引用都由 Ratchet 统一管理。

Example

综合示例：默认块使用深度 3，仅把 `remark` 从 `prop` 共享族中拆出并设为深度 1，再创建深度 2 的 `algorithm`。

```
#let blocks = set-countblock-depth(cb, "rmk", 1, detach: true)
#let blocks = add-countblock(blocks, "alg", "Algorithm", yellow, depth: 2)
#let remark = countblock.with("rmk", blocks)
#let algorithm = countblock.with("alg", blocks)

#show: scripst.with(
  // ...
  countblocks: blocks,
  cb-counter-depth: 3,
)
```

把字典和封装函数放在 `#show: scripst.with(...)` 之前即可。

III.7 一些其他的块

III.7.1 空白块

此外，Script 还提供了这样的无标题的块，你可以自定义颜色来使用。

例如

```
#blankblock(color: color.red)[
  这是一个红色的块。
]
```

这是一个红色的块。

III.7.2 证明与■（证明结束）

```
#proof[
  这是一个证明。
]
```

Proof.

这是一个证明。

■

这提供一个简单的证明环境，以及证毕符号。

III.7.3 解答

```
#solution[
  这是一个解答。
]
```

Solution.

这是一个解答。

这提供一个简单的解答环境。

III.7.4 分隔符

```
#separator
```

可以使用 `#separator` 函数来插入一个分隔符。

III.8 其他格式调整

III.8.1 数学公式相关

III.8.1.1 引用编号

为数学公式调节了引用

$$\nabla^2 = \frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2} + \frac{\partial^2}{\partial z^2} \quad (3.9.1)$$

对 (3.9.1) 的引用，格式不再是“式 (3.9.1)”而是括号的形式。

III.8.1.2 cases 环境下的数学公式

`typst` 原本的 `cases` 环境在数学公式中使用，所有公式会显示成行内公式的形式。这有时候会导致美观性的问题。

$$u(x, t) = \begin{cases} \sum_{i=1}^{\infty} \frac{4}{n\pi} \sin\left(\frac{n\pi x}{L}\right) e^{-\left(\frac{n\pi}{L}\right)^2 \alpha t} & 0 < x < L \\ 0 & \text{otherwise} \end{cases} \quad (3.9.2)$$

`scripst` 提供了一个新的 `cases` 环境，可以在数学公式中使用。它的用法与原本的 `cases` 环境相同，如下所示：

```
$
u(x,t) = cases(
  gap: #1em,
  sum_(i=1)^oo 4/(n pi) sin((n pi x)/L) e^(-(n pi)/L)^2 alpha t) " "&
0<x<L,
  0 " " &"otherwise"
)
$
```

$$u(x, t) = \begin{cases} \sum_{i=1}^{\infty} \frac{4}{n\pi} \sin\left(\frac{n\pi x}{L}\right) e^{-\left(\frac{n\pi}{L}\right)^2 \alpha t} & 0 < x < L \\ 0 & \text{otherwise} \end{cases} \quad (3.9.3)$$

这是鉴于使用时，通常都需要展示比较详细的公式，所以在 `cases` 环境中，`scripst` 默认会将所有的公式都显示成行间公式的形式。

Note

如果你还想使用原来的 `math.cases`，可以使用

```
$
u(x,t) = #math.cases(
  $sum_(i=1)^oo 4/(n pi) sin((n pi x)/L) e^(-(n pi)/L)^2 alpha t) " "&
0<x<L$,
  $0 " " &"otherwise"$,
  gap: 1em
)
$
```

来使用。

IV 结语

上文展示了 Scripst 的使用方法，以及模板的参数说明和效果展示。

希望这篇文档能够帮助你更好地使用 typst 和 Scripst。

也欢迎你为 Scripst 提出建议、改善方法及贡献代码。

感谢您对 typst 和 Scripst 的支持！